

Q. Explain Definition of Quality, Quality Assurance, Quality Control, Difference between QA and QC, Software Quality Assurance Challenges in short.

Definition of Quality

Quality is the degree to which a product or service meets the specified requirements and customer expectations.

Quality Assurance (QA)

QA is a process-oriented approach that focuses on preventing defects in the development and production stages through planned and systematic activities.

Quality Control (QC)

QC is a product-oriented approach that focuses on identifying and correcting defects in the finished product through inspection and testing.

Difference between QA and QC

Aspect	Quality Assurance (QA)	Quality Control (QC)
Focus	Process	Product
Objective	Prevent defects	Detect defects
Method	Proactive (process improvement)	Reactive (inspection/testing)
Responsibility	Everyone involved in process	Usually testers or quality inspectors
Examples	Audits, process checklists	Product testing, inspections

Software Quality Assurance (SQA) challenges:

1. Incomplete or Changing Requirements

- Requirements may be unclear, incomplete, or frequently changing, making it hard to define test criteria and ensure quality.

2. Lack of Time and Resources

- QA teams are often under pressure to meet tight deadlines with limited personnel, tools, or time, leading to insufficient testing.

3. Inadequate Test Coverage

- Not all code paths or scenarios are tested, especially in complex systems, leading to undetected bugs.

4. Poor Communication

- Gaps in communication between developers, testers, and stakeholders can cause misunderstandings about features, priorities, or issues.

5. Test Environment Issues

- A test environment that doesn't match the production environment can lead to inaccurate results or missed bugs.

Q. explain software Quality Assurance, SQA Planning & Standards (ISO 9000) in short

Ans.

Software Quality Assurance (SQA)

SQA is a set of activities designed to ensure that software processes and products meet quality standards. It includes process monitoring, auditing, testing, and improving practices throughout the software development lifecycle to prevent defects.

SQA Planning

SQA Planning involves creating a structured approach to ensure quality in a project. It includes:

- Defining quality goals
- Choosing tools and methods
- Assigning responsibilities
- Setting milestones and metrics
- Planning reviews and audits

The goal is to integrate quality activities early and consistently in the development process.

ISO 9000 Standards

ISO 9000 is a family of international standards for quality management systems. Key points:

- Focuses on customer satisfaction and continuous improvement
- Defines principles like process approach, leadership, and evidence-based decisions
-

Q. explain Software Reliability & Reliability Measurement Factors: ROCOF, MTTF, MTTR, MTBF, POFOD, Availability in short

Ans.

Software Reliability

Software reliability is the probability that software will function without failure under specified conditions for a given period of time. It's a key aspect of software quality.

Reliability Measurement Factors

1. ROCOF (Rate of Occurrence of Failure)
 - Number of failures per unit time
 - Lower ROCOF = More reliable software
2. MTTF (Mean Time To Failure)
 - Average time between system startup and the first failure
 - Higher MTTF = Better reliability
3. MTTR (Mean Time To Repair)
 - Average time taken to repair a failure
 - Lower MTTR = Faster recovery
4. MTBF (Mean Time Between Failures)
 - Average time between two consecutive failures
 - $MTBF = MTTF + MTTR$
 - Higher MTBF = More reliable system
5. POFOD (Probability of Failure on Demand)

- Likelihood that software will fail when a service request is made
- Used in systems like safety-critical or emergency systems
- Lower POFOD = Higher reliability

6. Availability

- The proportion of time the system is operational and available
- $Availability = MTTF / (MTTF + MTTR)$
- Expressed as a percentage (e.g., 99.9%)

Q. explain Building Blocks of SQA, Software Quality factors

Ans.

Building Blocks of SQA

1. Software Engineering Standards
 - Guidelines and best practices (e.g., ISO, IEEE) to ensure quality processes.
2. Project Monitoring & Control
 - Tracking progress, identifying risks, and applying corrective actions.
3. Software Testing
 - Activities to detect defects through planned test cases.
4. Reviews & Audits
 - Inspections and evaluations of code, documents, and processes.
5. Configuration Management
 - Managing changes in software to maintain integrity and traceability.
6. Defect Tracking & Prevention

- Identifying, logging, analyzing, and eliminating root causes of defects.

7. SQA Metrics & Measurement

- Using measurable indicators (like defect density, test coverage) to assess quality.

Software Quality Factors (based on McCall's and ISO 9126 models)

1. **Correctness** – Software performs required functions correctly.
2. **Reliability** – Software runs without failure under given conditions.
3. **Efficiency** – Optimal use of system resources (CPU, memory, etc.).
4. **Integrity** – Protection from unauthorized access or data corruption.
5. **Usability** – Easy to understand, learn, and operate.
6. **Maintainability** – Easy to fix bugs or make updates.
7. **Flexibility** – Adaptable to changes.
8. **Portability** – Works across different platforms.
9. **Reusability** – Components can be reused in other applications.
10. **Interoperability** – Works well with other systems.

Q. Definition & Objectives of Testing

Ans.

Definition of Testing (in Software)

Testing is the process of evaluating a software system to detect defects and ensure it meets the specified requirements.

Objectives of Testing

1. **Detect Defects** – Identify bugs or errors in the software.
2. **Ensure Quality** – Verify that the software works as intended.

3. **Validate Requirements** – Confirm the software meets user needs and specs.
4. **Improve Confidence** – Build trust in the software's reliability and performance.
5. **Prevent Defects** – Help find root causes to avoid future issues.
6. **Support Maintenance** – Ensure changes or updates don't break functionality (regression testing).

Q. explain Role of testing and its effect on quality in short and easy language

Ans.

Role of Testing in Software Development

Testing plays a key role in making sure software works correctly and meets user needs. It helps find and fix problems before the software is released.

Effect of Testing on Quality

1. **Improves Quality** – By finding bugs early, testing helps deliver a more stable and reliable product.
2. **Ensures Functionality** – Confirms the software behaves as expected.
3. **Builds User Trust** – Quality-tested software is more likely to satisfy users.
4. **Saves Time and Cost** – Fixing bugs early is cheaper and faster than fixing them after release.
5. **Supports Continuous Improvement** – Helps identify weak areas in the process and improve future development.

Q. explain Economics of Testing

Ans.

Economics of Testing (in short)

Economics of Testing refers to the cost-benefit balance of software testing—how much to invest in testing versus the cost of defects if left unfound.

Key Points

1. **Early Testing Saves Money**

- Fixing bugs in early stages (like design) is much cheaper than fixing them after release.

2. Cost of Quality

- Includes prevention cost (e.g., training), detection cost (e.g., testing), and failure cost (e.g., fixing bugs).

3. Risk vs. Investment

- Testing helps reduce risk, but over-testing can waste time and money; a balance is needed.

4. Defect Cost Curve

- The later a defect is found, the more expensive it is to fix.

In short: Smart testing is a cost-effective way to avoid expensive problems later and improve software quality.

Q. explain Seven Testing Principles in short and easy explanation

Ans.

1. Testing shows presence of defects

Testing can show that bugs exist, but it can't guarantee there are no bugs at all.

2. Exhaustive testing is impossible

You can't test everything—there are too many possible inputs and paths. So, focus on the most important areas.

3. Early testing saves time and money

Start testing early (e.g., during design) to catch problems before they become expensive to fix.

4. Defect clustering

Most bugs are found in a few modules. Focus testing where problems are most likely.

5. Pesticide paradox: Running the same tests repeatedly won't find new bugs. You need to update and improve your tests regularly.

6. Testing is context dependent

How you test depends on the type of software (e.g., testing for a game vs. a medical app is very different).

7. Absence-of-errors fallacy

Even if a program has no bugs, it's useless if it doesn't meet user needs or requirements.

Q. Software Testing Life cycle

Ans.

Software Testing Life Cycle (STLC) – In Short

The Software Testing Life Cycle (STLC) is a series of phases followed during the testing process to ensure quality and thoroughness.

Phases of STLC

1. Requirement Analysis

- Understand what needs to be tested
- Identify testable requirements

2. Test Planning

- Define scope, strategy, resources, and schedule
- Prepare a test plan document

3. Test Case Design

- Write detailed test cases and test data
- Ensure coverage of all scenarios

4. Test Environment Setup

- Prepare hardware/software for testing
- Simulate the real-world environment

5. Test Execution

- Run test cases
- Log defects if any

6. Defect Reporting & Tracking

- Report bugs to developers
- Track them until resolved

7. Test Cycle Closure

- Final reporting and analysis
- Learn lessons for future projects

In short: STLC is a structured way to plan, prepare, run, and close testing activities for better quality software.

Q. explain Validation & Verification Concepts - V Model and W Model

Ans.

Validation & Verification Concepts

- **Verification:** Ensures the software was built correctly (i.e., it meets specifications and requirements).
- **Validation:** Ensures the right software was built (i.e., it meets user needs and expectations).

V-Model (Verification and Validation)

The V-Model is a development process where each phase of development corresponds directly to a phase of testing.

- **Left side (Verification):** Focuses on defining requirements, design, and architecture.
- **Right side (Validation):** Focuses on testing the system to ensure it meets the specifications and user needs.

Key point: As you design the software, you simultaneously define how it will be tested. Verification activities (like design reviews) are paired with corresponding validation activities (like user acceptance testing).

W-Model (Extended V-Model)

The W-Model is similar to the V-Model but adds an additional emphasis on integration testing and validation across different stages.

- It highlights the need for early testing and continuous validation throughout the development process, including the integration phase.

Key point: The W-Model extends the V-Model by emphasizing the importance of testing not just at the end, but throughout development, especially during integration.

In short:

- The V-Model connects each development phase with a testing phase.
- The W-Model extends this by focusing on continuous validation and integration testing.

Q. explain Levels of Testing- Unit (Component) Testing Integration Testing, System Testing, User Acceptance Testing (UAT)

Ans.

Unit (Component) Testing

- **Purpose:** Test individual units or components of the software (like a single function or module).
- **Focus:** Verifying that each component works as expected in isolation.
- **Performed by:** Developers.
- **Example:** Testing a function that adds two numbers.

Integration Testing

- **Purpose:** Test how different modules or components interact with each other.
- **Focus:** Identifying issues when components are integrated.
- **Performed by:** Developers or testers.
- **Example:** Checking if the user login component works correctly with the database.

System Testing

- **Purpose:** Test the entire system as a whole to ensure all components work together as expected.
- **Focus:** Verifying the complete software system against the specified requirements.
- **Performed by:** Testers.

- Example: Testing the full application to see if all features (login, shopping cart, checkout) work together.

User Acceptance Testing (UAT)

- Purpose: Verify the software meets the user's needs and business requirements.
- Focus: Ensuring the system is ready for deployment.
- Performed by: End users or stakeholders.
- Example: A business team tests if the application supports their day-to-day operations as expected.
- Unit Testing: Tests individual parts.
- Integration Testing: Tests interactions between parts.
- System Testing: Tests the whole system.
- UAT: Tests for user satisfaction and readiness.

Q. explain Functional testing (Black-box) Non-functional testing (Testing of software product characteristics) Structural testing (White-box) Testing related to changes – Confirmation (Re-testing) and Regression Testing

Ans.

Functional Testing (Black-box Testing)

- Purpose: Focuses on what the software does (its functionality).
- Approach: Tests the system based on its requirements and specifications without knowing the internal code.
- Example: Testing if a login form accepts valid credentials and rejects invalid ones.
- Type of Testing: Black-box (No knowledge of internal workings).

Non-functional Testing (Testing of Software Product Characteristics)

- Purpose: Focuses on how well the software performs, rather than its functionality.

Characteristics Tested:

- Performance: How fast the system works.
- Security: How safe the system is from threats.
- Usability: How easy it is to use the system.
- Compatibility: How well the system works on different devices or platforms.
- Example: Testing how many users can log in simultaneously (performance testing).

Structural Testing (White-box Testing)

- Purpose: Focuses on the internal workings of the software (i.e., code, logic, and structure).
- Approach: Tests the internal structure of the application by examining the code.
- Example: Testing code paths, loops, or conditions to ensure proper execution.
- Type of Testing: White-box (Requires knowledge of internal code).

Testing Related to Changes

- Confirmation Testing (Re-testing):
 - Purpose: Verify that a defect reported earlier has been fixed.
 - Example: After fixing a bug, re-run the same test to confirm the issue is resolved.
- Regression Testing:
 - Purpose: Ensure that new changes or fixes don't break the existing functionality of the software.
 - Example: After adding a new feature, test existing features to make sure they still work as expected.

In short:

- Functional Testing: Tests what the software does (black-box).

- **Non-functional Testing:** Tests characteristics like performance and security.
- **Structural Testing:** Tests how the software works internally (white-box).
- **Confirmation Testing:** Verifies if a specific defect is fixed.
- **Regression Testing:** Ensures new changes don't affect existing functionality.

Q. explain Concept of Smoke testing and Sanity Testing

Ans.

Concept of Smoke Testing and Sanity Testing

Both Smoke Testing and Sanity Testing are important types of preliminary testing performed to ensure that the software is stable enough for further testing. While they are similar in some ways, they have key differences.

Smoke Testing

- **Purpose:** To verify that the basic functionalities of the software are working after a new build or release.
- **Approach:** It involves running a few basic test cases to check whether the application is stable enough to proceed with further detailed testing.
- **Scope:** Smoke testing is usually high-level and broad, covering the main functionality, not deep testing.
- **Example:** Checking if the software starts, if the login page loads, if basic actions (e.g., adding an item to a cart) work.

Sanity Testing

- **Purpose:** To verify that specific functionality or bug fixes are working correctly after changes or updates.
- **Approach:** It focuses on a narrower scope and is done after receiving a new build with minor changes or bug fixes. It ensures that the particular issue has been addressed.

- **Scope:** Sanity testing is focused on testing specific areas of the software that were changed or updated.
- **Example:** After fixing a login bug, testing only the login functionality to ensure the fix works without checking the entire system.
- **Key Differences Between Smoke and Sanity Testing**

Aspect	Smoke Testing	Sanity Testing
Purpose	Check if the application is stable enough for further testing.	Check if specific functionalities or bug fixes are working.
Scope	Broad, covering major functionalities.	Narrow, focused on specific areas or changes.
When Performed	Performed after a new build or release.	Performed after specific changes or fixes.
Test Type	High-level, broad testing.	Focused and specific testing.
Depth of Testing	Shallow testing.	Deeper testing on the specific functionality.

Q. explain Non-Functional Testing Types – Performance (Load & Stress) Usability Maintainability Portability Security Localization & Internationalization

Ans.

Performance Testing

- **Purpose:** Evaluate the software's performance under various conditions.
- **Types:**
 - **Load Testing:** Measures the software's ability to handle expected user load. It checks whether the system can support the anticipated number of users and transactions.

- Example: Testing how many users can log in simultaneously without performance issues.
- Stress Testing: Tests the software beyond its normal capacity to determine its breaking point. This helps to understand how the system behaves under extreme conditions.
 - Example: Simulating thousands of users logging in at once to see how the system handles overload.

Usability Testing

- Purpose: Ensure that the software is easy to use and intuitive for end users.
- Focus: Evaluate the user interface (UI) and user experience (UX).
- Example: Testing if users can easily navigate the application without confusion.

Maintainability Testing

- Purpose: Assess how easy it is to maintain the software, including fixing bugs and adding new features.
- Focus: Evaluate how easy it is to update the software, understand its code, and fix issues.
- Example: Checking how easy it is to modify or update a module without affecting other parts of the system.

Portability Testing

- Purpose: Ensure that the software can work on different platforms, devices, or environments.
- Focus: Test how easily the software can be transferred, installed, and run in different environments (e.g., different operating systems, browsers).

- Example: Testing an application on both Android and iOS to ensure it works properly on both platforms.

Security Testing

- Purpose: Identify vulnerabilities in the software to ensure it is secure and protected against threats.
- Focus: Test for common security issues like data breaches, unauthorized access, and other vulnerabilities.
- Example: Testing for SQL injection, cross-site scripting (XSS), and ensuring proper encryption of sensitive data.

Localization & Internationalization Testing

- Purpose: Ensure the software works well for different languages and regions.
- Localization Testing: Verifies that the software is adapted to a specific locale, including language, date formats, currency, etc.
 - Example: Testing if the application correctly displays French language text and regional currency for users in France.
- Internationalization Testing: Ensures that the software is designed to support multiple languages and regions without requiring major changes.
 - Example: Checking if the system can support adding more languages without affecting core functionality.

In Short:

- Performance Testing: Tests how the system performs under load and stress.
- Usability Testing: Tests if the system is easy to use.
- Maintainability Testing: Tests how easy it is to maintain and update the system.

Q. explain Static Techniques – Review Review Process (Informal & Formal) Technical or Peer Review Walkthrough Inspection in short

Ans.

Static Techniques in Software Testing

Static testing involves evaluating the software without executing it. The goal is to identify issues in the code, requirements, design, and documentation early in the development cycle.

Review

- Purpose: A general term for evaluating and assessing software artifacts like code, design, and requirements to detect defects early.
- Focus: Identifying issues, inconsistencies, and improvements in documents, designs, or code.

Review Process (Informal & Formal)

Informal Review

- Purpose: A casual review where the author of the work and one or more peers check for defects. There is no structured process or documentation.
- Example: A developer quickly checks the code before committing it to the repository.

Formal Review

- Purpose: A structured and planned review process, typically involving multiple participants, defined roles, and detailed documentation. It follows a formal process to ensure thorough evaluation.
- Example: A detailed meeting where all team members review design or code, and defects are formally logged.

Technical or Peer Review

- Purpose: A review where technical experts (often colleagues or peers) evaluate the software for correctness, adherence to standards, and potential issues.

- Focus: Detailed technical assessment of code or design, identifying bugs, improvements, and best practices.
- Example: A senior developer reviewing a junior developer's code for performance, security, and standards compliance.

Walkthrough

- Purpose: A meeting where the author of the work (like a document, code, or design) presents it to a group, and participants provide feedback.
- Focus: A step-by-step walkthrough of the work to understand and find issues, often informally.
- Example: A developer walks through their code to explain the logic to the team and receives feedback.

Inspection

- Purpose: A formal and detailed review of the software artifacts (requirements, design, or code) to detect defects, deviations, and improve quality.
- Focus: A structured approach where each participant plays a specific role (like a moderator, reader, recorder) to thoroughly inspect the product.
- Example: A group of developers and testers systematically review the codebase for errors, design flaws, and ensure standards are met.

In Short:

- Review: General process of evaluating software artifacts.
- Informal Review: Casual and unstructured review by peers.
- Formal Review: Structured review with defined roles and documentation.
- Technical or Peer Review: In-depth review by technical experts or peers.
- Walkthrough: Author presents the work to the group for feedback.

- **Inspection:** Formal, detailed, and structured review process.

Q. explain Static Techniques – Static Analysis
Static Analysis by Tools (Automated Static Analysis) in short

Ans.

Static Techniques in Software Testing – Static Analysis

Static analysis is the process of examining the software without executing it. This technique involves analyzing the code, requirements, design, or documentation to find errors or potential improvements at an early stage of development.

Static Analysis

- **Purpose:** The goal is to detect defects, code quality issues, and vulnerabilities by analyzing the software's source code or documents, without running the program.
- **How it's done:** The code is reviewed for compliance with coding standards, potential bugs, security issues, or areas where the code could be improved.
- **Focus Areas:**
 - **Code quality:** Identifying inefficient or complex code.
 - **Code violations:** Checking for coding standard violations.
 - **Security vulnerabilities:** Detecting potential security flaws like buffer overflows or SQL injections.
- **Example:** Checking if there are unused variables or redundant code blocks that can be optimized.

Static Analysis by Tools (Automated Static Analysis)

- **Purpose:** Using automated tools to perform static analysis on the software's codebase to detect errors and quality issues automatically.

- **How it's done:** Special tools analyze the source code or documents without executing the program. These tools can identify patterns, potential bugs, or areas that need improvement based on predefined rules.

- **Benefits:**
 - **Efficiency:** Automated tools can quickly analyze large codebases.
 - **Consistency:** Ensures the same analysis is applied every time, reducing human error.
 - **Early detection:** Helps detect bugs and issues early in the development cycle, even before the code is compiled or run.

- **Example Tools:**
 - **SonarQube:** Checks for code quality, security vulnerabilities, and code smells.
 - **Checkstyle:** Checks for adherence to coding standards (e.g., indentation, naming conventions).
 - **FindBugs:** Detects potential bugs in Java code.

Q. explain Decision Table Testing

Ans.

Decision Table Testing – In Short

Decision Table Testing is a black-box testing technique used to test systems where different combinations of inputs can produce various outputs. It helps ensure that the system handles all possible input conditions appropriately.

Key Concepts of Decision Table Testing

1. **Decision Table:** A table that defines combinations of input conditions and their corresponding actions or outcomes. It's often used when there are multiple conditions that can affect the result.
2. **Components of a Decision Table:**
 - **Conditions (Inputs):** The inputs or situations that can influence

the decision (e.g., conditions like "Age > 18", "Has Membership").

- **Actions (Outputs):** The result or output based on the conditions (e.g., "Eligible for Discount", "Eligible for Special Offer").
- **Rules:** Each rule represents a unique combination of conditions and specifies the corresponding action.

3. How it Works:

- The decision table is divided into conditions and actions.
- Each combination of conditions is tested to ensure that the system behaves as expected under all possible scenarios.
- Each rule in the table represents a unique scenario for testing.

Benefits:

- **Clear and organized:** Helps to map complex logical conditions into an easy-to-understand format.
- **Comprehensive:** Ensures all possible combinations of inputs and outputs are tested.
- **Systematic:** Reduces the chances of missing a condition or combination.

Q. explain State Transition Testing

Ans.

State Transition Testing – In Short

State Transition Testing is a black-box testing technique used to test the behavior of a system where the output depends not only on the current input but also on the previous state of the system. It is useful for systems that have multiple states and transitions between those states based on different inputs.

Key Concepts of State Transition Testing:

1. **States:** Different conditions or statuses in the system. For example, a traffic light could have the states: Red, Yellow, and Green.

2. **Transitions:** The movement from one state to another triggered by an event or input. For example, the traffic light can transition from Green to Yellow when the timer expires.
3. **Events:** Inputs that cause a transition between states. These can be user actions or system events. For example, a button click could trigger a state change.
4. **Actions:** Activities or behaviors that occur when a transition happens. For example, when the traffic light changes from Red to Green, it turns on the green light.

How it Works:

- **Identify States:** List all possible states of the system.
- **Identify Transitions:** Determine the events that cause a state transition.
- **Create State Transition Diagram:** A diagram that shows the system's states and the transitions between them.
- **Create Test Cases:** Based on the diagram, create test cases that ensure the system behaves correctly when transitioning from one state to another.

Benefits:

- **Clear and structured:** Helps in understanding how the system behaves in different states and under different conditions.
- **Comprehensive:** Ensures all possible transitions between states are tested.
- **Systematic:** Helps detect any incorrect or missing transitions in the system.

Q. explain Test Design Techniques -White Box Testing Techniques

Ans.

Test Design Techniques – White Box Testing Techniques

White Box Testing (also known as Clear Box Testing or Structural Testing) focuses on testing the internal structure of the software. It involves

looking at the code and design of the application to create test cases that ensure proper execution of the internal logic, code paths, and conditions.

Statement Coverage

- **Purpose:** Ensures that every statement in the code is executed at least once during testing.
- **Focus:** Verifying that all lines of code are run during test execution.
- **Example:** If there's a function with multiple lines of code, the test should ensure that each line is covered by at least one test case.

Branch Coverage

- **Purpose:** Ensures that every possible branch (i.e., decision point) in the code is tested. This involves ensuring that each decision (like if or case statements) takes both true and false branches at least once.
- **Focus:** Testing the decision-making paths in the code.
- **Example:** In an if-else statement, both the true path and the false path should be tested.

3. Path Coverage

- **Purpose:** Ensures that every possible path through the code is tested. This includes all combinations of branches and conditions.
- **Focus:** Testing all the possible routes through the code, from start to end, considering loops, branches, and decisions.
- **Example:** If a function has multiple loops and conditions, path coverage would ensure that every possible combination of those loops and conditions is tested.

Q. explain McCabe's Cyclomatic Complexity Metric in short

Ans.

McCabe's Cyclomatic Complexity Metric – In Short

McCabe's Cyclomatic Complexity is a software metric used to measure the complexity of a program based on its control flow. It helps quantify the number of linearly independent paths through a program's source code. This metric is useful in determining the testability and maintainability of a program.

Key Concepts of Cyclomatic Complexity:

1. **Control Flow Graph (CFG):** The program's control flow is represented as a graph, where:
 - Nodes represent instructions or blocks of code.
 - Edges represent the flow of control between the blocks.
2. **Cyclomatic Complexity (V(G)):** The formula to calculate cyclomatic complexity is:

$$V(G) = E - N + 2P \quad V(G) = E - N + 2P$$

Interpretation:

- A higher cyclomatic complexity value indicates a more complex program with more paths, which may require more testing and maintenance.
- A lower value indicates a simpler program, easier to test and maintain

In Short:

- McCabe's Cyclomatic Complexity measures the complexity of a program based on its control flow.
- The higher the cyclomatic complexity, the more complex the program is, and more testing is required.
- Formula: $V(G) = E - N + 2P$

A higher complexity suggests the need for more detailed testing and increased maintainability efforts.

Q. explain Test Design Techniques in short

Ans.

Test Design Techniques – In Short

Test design techniques are methods used to create effective and efficient test cases for software applications. These techniques help ensure comprehensive coverage and maximize defect detection. There are several types of test design techniques, and they are typically classified into black-box and white-box techniques.

Black-Box Testing Techniques

These techniques focus on testing the functionality of the software based on its specifications, without any knowledge of the internal code.

a. Equivalence Partitioning

- Purpose: Divides input data into equivalence classes, where all values in a class are expected to be treated the same by the system.
- Example: If an input field accepts ages between 18 and 60, then valid equivalence classes are ages 18-60, and invalid equivalence classes could be less than 18 or greater than 60.

b. Boundary Value Analysis (BVA)

- Purpose: Focuses on testing the boundaries of equivalence classes, as errors are often found at boundary values.
- Example: For an age input between 18 and 60, test values at the boundaries like 17, 18, 60, and 61.

c. Decision Table Testing

- Purpose: Tests combinations of different input conditions (i.e., decision-making rules) to verify the system handles all possible scenarios.
- Example: A decision table for granting discounts based on membership status and purchase amount.

d. State Transition Testing

- Purpose: Tests how the system transitions from one state to another, based on inputs or events.
- Example: Testing the different states of a traffic light (Red, Yellow, Green) and ensuring correct transitions.

e. Error Guessing

- Purpose: Involves using experience and intuition to predict where errors might occur, then designing test cases to target these areas.
- Example: Guessing that an error might occur when entering special characters in a form field, then testing that scenario.

White-Box Testing Techniques

These techniques focus on testing the internal structure of the code, including logic and path coverage.

a. Statement Coverage

- Purpose: Ensures that every line of code is executed at least once during testing.
- Example: Testing all functions to verify that every statement in the program is executed.

b. Branch Coverage

- Purpose: Ensures that each possible decision or branch in the code is tested, i.e., both true and false branches are exercised.
- Example: Testing an if-else statement to ensure both branches (true and false) are covered.

c. Path Coverage

- Purpose: Ensures that all possible paths through the program (including combinations of decisions) are tested.
- Example: Testing different possible routes through a program with multiple conditions and loops.

d. Condition Coverage

- Purpose: Ensures that each individual condition in a decision statement is tested for both true and false outcomes.
- Example: For a condition like if ($x > 10$ && $y < 5$), test $x > 10$ true/false and $y < 5$ true/false.

e. Loop Coverage

- Purpose: Ensures that all loops are tested for zero, one, and multiple iterations.
- Example: Testing a for loop with 0 iterations, 1 iteration, and a high number of iterations.

Experience-Based Techniques

These techniques are based on the experience of the tester, often using intuition or prior knowledge to design tests.

a. Exploratory Testing

- Purpose: Testers explore the application to learn about its behavior and design tests based on their findings, often without predefined test cases.
- Example: A tester interacts with the application, searching for unexpected behaviors or bugs.

b. Ad-hoc Testing

- Purpose: Similar to exploratory testing, but with no structured approach or planning. It relies heavily on the tester's experience.
- Example: Randomly testing features without a defined strategy, often to quickly find bugs.

Q. explain exploratory Testing in short

Ans.

Exploratory Testing – In Short

Exploratory Testing is a type of testing where the tester actively explores the application while learning about it. Instead of following predefined test cases, the tester uses their knowledge, creativity, and intuition to discover defects and unexpected behaviors during testing.

Key Features of Exploratory Testing:

1. **Learning while Testing:** Testers learn about the software as they test, which helps them come up with new test ideas based on their observations.
2. **No Predefined Test Cases:** Unlike traditional testing, where specific test cases are created in advance, exploratory testing allows the tester to ad hoc explore different parts of the software.
3. **Flexibility:** The tester can adjust the testing strategy based on what they discover as they interact with the software.
4. **Simultaneous Test Design and Execution:** In exploratory testing, the tester designs and runs tests at the same time, making it a more dynamic and adaptive process.

Q. explain Test Organization- Roles & Skills of Tester, Test Lead, Test Manager

Ans.

Test Organization – Roles & Skills of Tester, Test Lead, Test Manager

In a software testing organization, different roles are defined to ensure testing is structured, effective, and aligned with project goals.

Tester

Role:

A Tester is responsible for executing test cases, finding defects, and reporting issues to ensure that the software works as expected. They

primarily focus on functional testing, non-functional testing, and exploratory testing

Test Lead

Role:

A Test Lead is responsible for overseeing the testing process, managing the testing team, and ensuring the successful delivery of test results. They also assist in test planning and test case creation.

Test Manager

Role:

A Test Manager is responsible for the overall management of the testing process within a project or organization. They focus on the big picture: managing resources, budgets, timelines, and ensuring that the testing aligns with the project's goals and quality standards.

In Short:

- **Tester:** Executes tests, finds defects, and reports issues. Requires attention to detail and technical skills.
- **Test Lead:** Manages the testing team, oversees the test process, and ensures successful test execution. Requires leadership and test planning expertise.
- **Test Manager:** Oversees the entire testing process, manages resources, budgets, and risks, and ensures alignment with business goals. Requires strong leadership, management, and strategic thinking skills.

Q. explain Test Planning- Test Plan as per IEEE 829 STANDARD TEST PLAN TEMPLATE

Ans.

The IEEE 829 Standard provides a structured template for creating a Test Plan, which outlines the scope, approach, resources, and schedule for testing a software product.

Test Plan Template (IEEE 829):

1. **Test Plan Identifier:** A unique identifier for the test plan.

2. **Introduction:** A brief overview of the testing objectives and scope.
3. **Test Items:** The software components or features to be tested.
4. **Features to be Tested:** Specific functionalities or requirements to be verified during testing.
5. **Features Not to be Tested:** Exclusions or functionalities that won't be tested.
6. **Test Approach:** High-level strategy for testing, including test levels (unit, integration, system, UAT) and testing types (functional, non-functional).
7. **Pass/Fail Criteria:** Criteria for determining whether a test is successful or not.
8. **Test Deliverables:** The documents and reports to be delivered (test cases, test results, defect logs).
9. **Testing Tasks:** Specific tasks to be performed (e.g., test case design, environment setup, execution).
10. **Environmental Needs:** Hardware, software, and network requirements for testing.
11. **Responsibilities:** Roles and responsibilities of team members involved in testing.
12. **Schedule:** Timeline and milestones for test activities.
13. **Risks and Contingencies:** Potential risks and backup plans.
14. **Approvals:** Sign-offs from stakeholders to approve the plan.

Q. explain Test Monitoring through -Test Log (IEEE 829: TEST LOG TEMPLATE) and Defect Density in very short

Ans.

Test Log (IEEE 829)

A Test Log is a record of all the test activities during the testing process. It captures detailed information about the execution of tests to track progress and identify issues.

Test Log Template (IEEE 829):

- **Test Log Identifier:** A unique identifier for the test log.
- **Test Items:** The items or components being tested.
- **Test Execution Date:** The date when the test was executed.
- **Tester:** The name of the person who executed the test.
- **Test Environment:** The environment in which the test was executed (e.g., hardware, software version).
- **Test Result:** The outcome of the test (pass/fail).
- **Defects Found:** Any defects or issues discovered during testing.
- **Comments:** Additional notes or observations during test execution.

Purpose: A Test Log helps track test progress, document issues, and provide transparency for stakeholders about the status of testing.

Defect Density

Defect Density is a metric that measures the number of defects in a software product relative to its size or complexity. It helps assess the quality of the software and identify areas that need attention.

Formula:

Defect Density = $\frac{\text{Number of Defects}}{\text{Size of the Software}}$
Defect Density = $\frac{\text{Size of the Software}}{\text{Number of Defects}}$

Where the size could be measured in lines of code (LOC), function points, or modules.

Q. explain Reporting Test Status (IEEE 829: TEST SUMMARY REPORT TEMPLATE)

Ans.

The Test Summary Report is a document used to summarize the results and status of testing. It provides stakeholders with an overview of the testing process, including what was tested, how it was tested, the outcomes, and any issues found. The IEEE 829 standard defines a template for creating this report.

A Test Summary Report provides an overview of the testing process, results, and issues. It includes:

1. **Test Summary:** High-level activities and tests executed.
2. **Test Results:** Pass/fail statistics, coverage, and outcomes.
3. **Defect Summary:** Number and severity of defects found.
4. **Deviations:** Any differences from the original test plan.
5. **Risks and Issues:** Any identified risks or unresolved problems.
6. **Conclusion:** Summary of testing results and software readiness.
7. **Approvals:** Sign-offs from stakeholders.
8. It helps communicate testing progress and software quality to stakeholders.

Q. explain Test Scenario, Test Suite, Test Cases (both Positive & Negative Test Cases, as per IEEE 829: TEST CASE SPECIFICATION TEMPLATE)

Ans.

Test Scenario

A Test Scenario is a high-level idea or condition to be tested.

- Example: "Login with valid and invalid credentials."

2. Test Suite

A Test Suite is a collection of test cases grouped together for a specific purpose or module.

- Example: A login test suite may contain test cases for valid login, invalid login, blank fields, etc.

3. Test Case

A Test Case is a set of actions and inputs used to verify a particular feature or function.

Defined as per IEEE 829: Test Case Specification Template, it includes:

- Test Case ID
- Test Description
- Input Data
- Expected Result
- Actual Result
- Status (Pass/Fail)

Types of Test Cases:

- **Positive Test Case:** Verifies the system works as expected with valid input.
 - *Example:* Enter correct username and password → Login successful.
- **Negative Test Case:** Verifies the system handles invalid input properly.
 - *Example:* Enter wrong password → Show error message.

Q. explain Defect Life Cycle

Ans.

The Defect Life Cycle (or Bug Life Cycle) is the process a defect goes through from identification to closure.

Key Stages:

1. **New** – Tester finds a defect and reports it.
2. **Assigned** – Defect is assigned to a developer for fixing.
3. **Open** – Developer starts working on the defect.
4. **Fixed** – Developer fixes the defect.
5. **Retest** – Tester verifies if the fix works.

6. **Closed** – Tester confirms the defect is resolved.
7. **Reopened** – If the issue still exists after fix.
8. **Rejected / Not a Bug** – If it's not a valid defect.
9. **Deferred** – Fix is postponed for a future release.
10. **Duplicate** – Same as an existing reported defect.

The **Defect Life Cycle** tracks a bug from **discovery to closure**, helping teams manage and resolve issues systematically.

Q. explain Types of Test tools –CAST(types, there purpose, benefits & risks)

Ans.

CAST tools help automate or support various software testing tasks.

Type	Purpose	Benefits	Risks
Test Management Tools	Plan, track, and manage test activities	Better organization, progress tracking	Tool learning curve
Test Execution Tools	Run test cases automatically	Faster execution, repeatability	May miss visual/UI issues
Defect Tracking Tools	Log and track bugs	Easy monitoring, better communication	Poor integration can delay reporting
Performance Testing Tools	Check system speed and load handling	Identify bottlenecks early	Costly and require expertise
Static Analysis Tools	Analyze code	Early defect detection	False positives, not all issues are found

Type	Purpose	Benefits	Risks
	without execution		
Test Data Generation Tools	Create test data automatically	Saves time, improves coverage	Can generate irrelevant/unrealistic data

Q. Testing tools 1.Selenium -WebDriver and TestNG 2. JMeter 3. Postman 4. ETL Testing Tool

Ans.

Selenium (WebDriver + TestNG)

- **Selenium WebDriver:** Open-source tool for automating web applications across browsers.
- **TestNG:** Java-based testing framework used with Selenium for managing test cases, assertions, and reporting.
- **Use:** UI Testing (Functional & Regression)
- **Benefits:** Cross-browser testing, automation of repetitive tasks.
- **Limitations:** Limited support for desktop/mobile apps, needs coding.

JMeter

- **Tool Type:** Performance & Load Testing.
- **Use:** Simulates multiple users to test web app performance under load.
- **Benefits:** Open-source, supports many protocols (HTTP, FTP, etc.).
- **Limitations:** Not ideal for UI testing, complex for beginners.

Postman

- **Tool Type:** API Testing Tool.
- **Use:** Testing RESTful APIs (send requests, validate responses).
- **Benefits:** Easy to use, supports automation with collections.

- **Limitations:** Focused only on API, not suitable for full UI testing.

ETL Testing Tool

- **ETL (Extract, Transform, Load) Testing:** Validates data movement from source to target in data warehouses.
- **Tools:** Informatica, Talend, QuerySurge, etc.
- **Use:** Data consistency, accuracy, and completeness checks.
- **Benefits:** Ensures reliable data migration and transformation.
- **Limitations:** Complex setup, depends on data knowledge.

Q. explain JIRA (Project Management) in short

Ans.

JIRA is a project management and issue-tracking tool developed by Atlassian, widely used in software development and Agile environments.

Key Features:

- **Issue Tracking:** Create, assign, and track bugs, tasks, or user stories.
- **Agile Boards:** Supports Scrum and Kanban boards for sprint planning and tracking.
- **Workflows:** Customizable steps for task progress (To Do → In Progress → Done).
- **Reporting:** Generates burndown charts, velocity reports, etc.
- **Integration:** Connects with tools like Confluence, Git, Jenkins.